SPHINCS+, SPHINCS-α, SPHINCS+C, and DGSP: A Comparative Implementation Study

Applied Cryptography Term Project Report

Thomas Chandler
z5479498

Justin Dy
z5480124

Lachlan Johnston
z5477322

Matthew Qiao
z5470659

April 24, 2026

Abstract

The NIST standardisation of SLH-DSA [17] has renewed interest in stateless hash-based signatures as a conservative foundation for post-quantum cryptography. Since standardisation, a growing body of work has proposed optimisations to the underlying SPHINCS+ construction, however, existing literature is yet to adapt these optimisations as a primitive for post-quantum group signature schemes such as DGSP. What remains unclear is how far these optimisations push the practical reach of SLH-DSA, and how the resulting schemes compare against alternative group-signature designs. This project presents a unified Python implementation and evaluation of SPHINCS+ alongside two signature-size optimisations from the recent literature, SPHINCS-α and SPHINCS+C, and the DGSP group signature scheme instantiated over each of these variants. We additionally perform parameter-space exploration to characterise the trade-offs between signature size, key generation cost, and signing time. Python was chosen to enable a single, readable codebase covering every variant, with absolute performance benchmarked against published C and Rust results from the original works. Our results quantify the cumulative improvement over the FIPS 205 baseline achievable by stacking these optimisations [INSERT HEADLINE NUMBER], and position DGSP against other post-quantum group signature schemes including DGMT and SiTH [INSERT COMPARISON]. We close with a discussion of how these gains compare against the lattice-based alternative ML-DSA (FIPS 204), and where hash-based approaches remain preferable.

Our goal is also to test different parameter sets and benchmark them against certain SHAKE results

Acknowledgements

We would like to formally acknowledge the core contributions made by the SPHINCS+ project team for their continued development in encouraging further optimisations of the SPHINCS+ scheme. The development of SPHINCS+ has allowed other groups of people to find methods to further optimise the ideas found in the initial paper. Furthermore, to all the SPHINCS+ optimisations whom we've taken inspiration from such as SPHINCS+C, SPHINCS+α, DGSP, we deeply and sincerely celebrate your work for it has underpinned the research as a whole. We would like to extend these acknowledgements to our lecturer and tutors who have given us understanding towards the field of cryptography as a whole and whose project ideas have become the impetus of this research and report.

To any researchers reading this report, we hope that the results of comparing the effects of these optimisations unto the practicality of SPHINCS+ has allowed for further insight into other possible avenues of development.

Contents

List of Figures

1 Introduction

Hash-Based Signature Schemes

Hash Based Signature Schemes have only recently gained a resurgence with the introduction of XMSS and SPHINCS in 2010 and 2015. The core of hash-based signature schemes is that their security relies firmly on the presumed difficulty of finding preimages for hash functions [17]. This project, in particular, focuses on the SPHINCS+ signature scheme which in essence is a signature scheme formed through the use of other hash-based signature schemes as components, forest of random subsets (FORS) and the eXtended Merkle Signature Scheme (XMSS) constructed through the hash-based one time signature scheme Winternitz One-Time Signature Plus (WOTSPLUS) [17].

Our Contributions

We have created an implementation of the Hash-Based Signature Scheme called SPHINCS+ in python along with some of its variations such as SPHINCS-α, SPHINCS+C and SPHINCS+ DGSP (SPHINCS+ as a group signature scheme). Through these different SPHINCS+ optimisations and varitions, we have allowed some variations to combine with each other, most notably DGSP with either SPHINCS+, SPHINCS+α and SPHINCS+C through the python code base. Through this, we allow for a deep comparison into the effects of SPHINCS+ optimisations as well as allow for a deeper and better understanding of the tradeoffs that each variation brings in. Through this understanding of tradeoffs, we allow for a deeper analysis as to the practically of the signatrue scheme's feasibility cost within some real life capacity.

Our use of coding within the code base of Python is to also allow for a deeper understanding as to the differences between SPHINCS+ instantiations when coded in different languages. We want to help understand how coding langauges can practically influence the feasibility of the signature scheme, especially in regards to Python which is known for not being the most memory efficient.

Requirements and Responsibilities

The implementation itself functions as a proof of concept and as a practical analysis of SPHINCS+ rather then a real life applicable standardised scheme. This implementation should not be used as is especially within the context of being a hash-based signature scheme to be used in protecting valuable data and privacy. This is moreso the case as it is implemented through Python which can be memory inefficient thus leading to delays which can cause practicality issues.

It can be transformed to allow for such things, but do keep in mind that the components of the signature scheme should not be allowed for use in a public interface and, that just like any other signature scheme, remains vulnerable to possible side-channel, fault attacks and other such possible avenues of attack. Any transformations of this codebase must, firstly, keep in mind that this codebase and the report are foundational-ly based upon the works of other people who have optimised SPHINCS+ as well as the pioneers of SPHINCS and SPHINCS+ and as such due diligence is necessary within the proper algorithms to ensure a succint, stable and secure transformation to a different coding language.

Important data in regards to the functioning of the signature scheme such that could potentially reveal any information about it such as the signature, public key and message must be destroyed after use. The protection and secrecy of the secret key is essential when using the system and no local copies be stored to ensure strict confidentiality of the system.

2 Background and Related Work

Hash-Based Digital Signature Schemes

Hash-based digital signatures trace back to the OTS construction of Lamport [15] and the tree-based many-time extension due to Merkle, both of which rely on the security of the underlying hash function rather than number-theoretic hardness assumptions. Unlike number-theoretic schemes whose security relies on the hardness of problems such as integer factorisation or discrete logarithm - both efficiently solvable by Shor's algorithm [18] - hash-based schemes reduce entirely to well-studied properties of the underlying hash function: one-wayness, second-preimage resistance, and distinct-function second-preimage resistance. These properties are assumed to hold against quantum attacks with at most a quadratic speedup via Grover's algorithm, making hash-based signatures among the most conservatively founded post-quantum constructions.

The core building block of all these schemes is the one-time signature (OTS), a primitive that allows a single key pair to sign exactly one message securely, with many-time schemes constructed by authenticating large collections of OTS key pairs via Merkle trees. The Winternitz OTS (WOTS), formalised in [7], improves upon Lamports scheme by reducing signature size through iterative hash chaining. A further refinement, WOTS+, introduced in [13], enhances security by incorporating randomness into the chaining process and now serves as the central OTS primitive in modern hash-based signature schemes such as XMSS and SPHINCS+.

Development of SPHINCS: XMSS [8] introduced the first practically deployable stateful hash-based signature scheme with tight security bounds, but its requirement that the signer maintain state across operations made it unsuitable for general use within the NIST post-quantum standardisation process.

SPHINCS was introduced by Bernstein et al. [3] as the first stateless hash-based signature scheme, eliminating the state requirement by authenticating a large collection of few-time signature (FTS) key pairs via a hypertree of Merkle tree signatures. The improved construction SPHINCS+ replaces the HORS few-time signature layer of SPHINCS with FORS (Forest of Random Subsets) and introduces tweakable hash functions to provide domain separation across all hash calls. The SPHINCS+ v3.1 specification [3] further consolidated the design and proposed parameter sets covering three security levels, each with a fast variant optimising for speed and a small variant optimising for signature size. Following NIST's selection, SPHINCS+ was standardised in 2024 [17] and compared to the co-standardised lattice-based ML-DSA (FIPS 204) [16], SPHINCS+ offers stronger security assumptions at the cost of larger signatures, positioning it as the conservative fallback should algebraic assumptions underlying lattice schemes be weakened.

Optimisations to the SPHINCS+ Construction

Despite the conservative security foundations, SPHINCS+ has attracted criticism for its large signature sizes relative to lattice-based alternatives. A line of work has sought to compress these signatures without sacrificing the scheme's minimal-assumption properties.

Zhang, Cui, and Yu [19] revisited the WOTS+ encoding at CRYPTO 2023, showing that the standard base-w encoding together with its checksum is suboptimal. They demonstrate that constant-sum encoding eliminates the need for a checksum entirely by mapping the message digest into a fixed-sum antichain—a set of vectors with fixed coordinate sum under which no element dominates another componentwise. This antichain property is precisely what prevents forgery, so no additional checksum chains are required.

The resulting scheme, SPHINCS-α, reduces the number of WOTS+ chains per signature and achieves signature size improvements of up to 10% in some parameter regimes while also reducing signing time. Importantly, constant-sum encoding enables parameter choices, particularly small Winternitz parameter w, that are infeasible under the standard checksum formulation due to integer overflow in the checksum computation.

A complementary approach was taken by Hülsing, Kudinov, Ronen, and Yogev [14], who introduced SPHINCS+C at IEEE Security & Privacy 2023. Rather than changing the message encoding, SPHINCS+C modifies what is signed: the signer searches for a short counter γ such that the hashed combination of the counter and message produces a digit string satisfying a fixed-sum condition *and* has z leading zero digits. The zero digits correspond to chains that carry no information, so those z chains are dropped from the signature entirely, while the fixed-sum condition removes the checksum chains. The counter is included in the signature so that the verifier can reproduce the digit string without repeating the search. The result is a new and more favourable size–time tradeoff curve: for a given signing cost, SPHINCS+C achieves strictly smaller signatures than the SPHINCS+ small variants, and the savings grow with both z and the Winternitz parameter w. The key insight is that, for certain predefined subsets of messages, WOTS+ and FORS signatures can be compressed with no degradation in security, enabling minor code changes relative to the base scheme. A more recent work by Abri and Katz [1] explores forced pruning as an alternative compression strategy, achieving further reductions in signature size, though this approach is not considered in the present study as we only picked it up late in this project.

Group Signature Schemes

The concept of group signatures was first introduced by Chaum and van Heyst [9] at EUROCRYPT 1991, who proposed a primitive allowing any member of a group to sign messages anonymously on behalf of the group, while a trusted authority retains the ability to open signatures and identify the signer in the event of a dispute. The notion of a group signature scheme, in which any member of a group may sign anonymously on behalf of the group while a designated authority retains the ability to revoke anonymity, was formalised by Bellare, Micciancio, and Warinschi [4] at EUROCRYPT 2003. Their work introduced the core security properties of anonymity, traceability, and non-frameability under formal game-based definitions, and showed that both properties are achievable from general trapdoor permutations. This model served as the foundational reference for subsequent constructions, though it addressed only static groups in which membership is fixed at setup. The extension to fully dynamic groups—where users may join and be revoked at any time without system-wide updates—was formalised by Bootle et al. [6] in 2020. Their work identified subtle gaps in earlier informal definitions and provided a unified security model that captures forward anonymity, non-frameability, and accountability in a dynamic setting, unifying previously incompatible models from the literature.

Post-Quantum Group Signature Schemes

The challenge of building group signatures from post-quantum assumptions has spurred a growing line of work, particularly constructions based on symmetric primitives rather than lattices or isogenies, as the former offer simpler security arguments under well-established assumptions. DGMT, proposed by Fadavi, Karati, Erfanian, and Safavi-Naini [12], is a fully dynamic group signature scheme built from symmetric-key primitives. It redesigns the earlier DGM construction (Buser et al., ESORICS 2019) to remove two practical limitations: the need for interaction with the group manager during signature verification, and the requirement to maintain an unmanageably large revocation state. However, DGMT supports at most 2^{15} group members, which limits its applicability in large-scale deployments.

Chen, Dong, Newton, and Wang [10] proposed Sphinx-in-the-Head (SiTH) at ACM CCS 2024, a post-

quantum group signature scheme built from symmetric primitives via MPC-in-the-head zero-knowledge proofs. SiTH introduces a novel hash-based group credential scheme rooted in a modified SPHINCS+ construction, making it MPC-friendly, and achieves post-quantum security in the Random Oracle Model under standard symmetric assumptions without relying on newly proposed algebraic hardness assumptions. The use of MPC-in-the-head, however, imposes a significant overhead on signature size and computational cost relative to schemes that directly leverage SPHINCS+ without the ZK layer.

DGSP, introduced by Fadavi, Azimi, Karati, and Jaques [11] and published at EUROCRYPT 2025, addresses the scalability limitations of both DGMT and SiTH. By building directly on the SPHINCS+ hypertree structure, DGSP supports up to 2^{60} users, requires negligible setup time, and produces group signatures that are only 3.03 to 4.93 times larger than the underlying SPHINCS+ signature. The scheme inserts an additional WOTS+ layer below the FORS trees to allow the group manager to issue user certificates, and relies on SPHINCS+ itself as a strong pseudorandom permutation for traceability. DGSP is currently the most scalable post-quantum group signature scheme based purely on symmetric-key primitives, and serves as the primary motivation for investigating how SPHINCS+ optimisations propagate into a group signature setting.

However, the limitations of the stateful one-time signatures underpinning DGSP represent a practical constraint. Because each user signing key is a WOTS+ key pair, each registered user may produce only a single group signature before their key is exhausted. A user wishing to sign again must interact with the group manager to obtain a fresh certificate, effectively requiring re-registration for every subsequent signing operation. This one-signature-per-registration restriction introduces non-trivial communication overhead between users and the group manager, and may be prohibitive in applications where individual users are expected to sign frequently or where interaction with the group manager cannot be assumed to be cheap or always available.

Positioning of This Work

Existing literature evaluates SPHINCS-α and SPHINCS+C as standalone improvements to the base scheme, and DGSP is benchmarked over the standard SPHINCS+ instantiation. No prior work has examined the effect of stacking these two independent optimisations, nor has any study instantiated DGSP over SPHINCS-α or SPHINCS+C to determine how the resulting group signature compares against DGMT and SiTH in practice. This project fills that gap through a unified Python implementation covering all four variants and a systematic parameter-space exploration.

3 Preliminaries

Hash Functions

All hash functions are instantiated using SHAKE-256 [**?**], an extendable-output function that allows a single primitive to serve every role in the SPHINCS+ framework. All functions produce n-byte outputs and are built on a common tweakable hash,

$$\mathsf{Thash}(\mathsf{PK.seed},\ \mathsf{ADRS},\ M) = \mathsf{SHAKE\text{-}256}(\mathsf{PK.seed} \,\|\, \mathsf{ADRS} \,\|\, M,\ n),$$

where ADRS is an address structure providing domain separation across components. The functions F, H, and T_ℓ used in the WOTS$^+$ chain, Merkle tree compression, and public key compression respectively are all specialisations of Thash. The pseudorandom functions PRF, $\mathsf{PRF}_{\mathsf{msg}}$, and H_{msg} are similarly instantiated directly from SHAKE-256, deriving secret key values, per-message randomisers, and message digests respectively.

Strong Pseudorandom Permutation

DGSP uses a strong pseudorandom permutation to support traceability. For each user identity id and certificate index j, the group manager computes

$$\rho_{id,j} = E(\mathsf{msk}_2,\ id \,\|\, j),$$

where E is instantiated as AES-128 in ECB mode via `pycryptodomex`. Opening a signature applies the inverse permutation E^{-1} to recover $(id,\ j)$ and identify the signer.

4 Implementation

This section describes our Python implementation of the two SPHINCS+ optimisations, SPHINCS-α and SPHINCS+C. This will cover the core idea behind each optimisation as well as the files added, the design decisions made and how they connect to the benchmarking results.

SPHINCS-α

Core Idea

Standard WOTS+ encodes a message by converting it to base-w digits $m_1, \ldots, m_{\ell_1}$ and appending a checksum $c = \sum_i (w - 1 - m_i)$, also represented in base w. This checksum occupies ℓ_2 additional chain positions, so the total number of chains is $\ell = \ell_1 + \ell_2$. This checksum is necessary to prevent forgery as without it, an adversary given a signiature could increment any m_i by running the corresponding chain forward a few steps.

SPHINCS-α eliminates the checksum by replacing the encoding scheme. Instead of appending a checksum, the message is mapped into the *constant-sum antichain* $C_s = \{c \in [w]^\ell : \sum_i c_i = s\}$ where $s = \lfloor \ell(w-1)/2 \rfloor$. Any two codewords in C_s are incomparable under the componentwise partial order, which is the property required to prevent forgery. because the sum is fixed by construction, no checksum field is needed and the total number of chains drops from $\ell_1 + \ell_2$ to ℓ_{cs}, where ℓ_{cs} is the smallest ℓ such that $|C_s| \geq 2^{8n}$.

Files Added

We split the implementation into three files following the same pattern as the base scheme:

- `WOTS/WOTS_Alpha.py` This is the OTS (one time signature) layer. Contains the DP table builder `compute_Dls`, the encoding function `cs_encode` (Algorithm 1 of [14]), its inverse `cs_decode`, the length solver `cs_len`, and the `WOTSAlpha` class.
- `params/sphincs_params_Alpha.py` This extends `SphincsParams` with a `cs_l` field.
- `sphincs/sphincs_Alpha.py` This is the top-level scheme. Wires `WOTSAlpha` through `XMSS_alpha` and `Hypertree_alpha` into a `SphincsAlpha` class with an identical `spx_keygen` / `spx_sign` / `spx_verify` interface to the base scheme.

Implementation Decisions

Message-to-index mapping. The paper specifies that the message digest is encoded into C_s via the bijection in Algorithm 1, but does not specify how to handle the fact that $|C_s|$ is not a power of two and the message space is $\{0,1\}^{8n}$. We treat the n-byte digest as a big-endian integer and reduce it by modulo $|C_s|$ before encoding. This introduces a small statistical bias on the order of $2^{8n}/|C_s| - 1$, which is negligible in practice since $|C_s| \approx 2^{8n}$ for all practical parameter sets.

Connection to Benchmarks

The signature size results for SPHINCS-α are reported in Table 4 of The Benchmarking Section. At $w = 16$, the saving is 32 bytes (2.1%) per signature, consistent with $d \cdot (\ell - \ell_{cs}) \cdot n = 2 \cdot 1 \cdot 16 = 32$ bytes. At $w = 256$, ℓ_2 is already minimal so there is no saving; however, at $w = 4$, which overflows entirely in the base scheme due to the checksum computation, SPHINCS-α produces valid signatures, which is the primary practical motivation for constant-sum encoding at small w.

SPHINCS+C

Core Idea

SPHINCS+C takes a different approach to eliminating the checksum. Rather than changing the encoding scheme, it changes what is signed. At signing time the signer searcher for a 32-bit counter γ such that $H(\text{PK.seed}\|\text{ADRS}\|\gamma\|M)$ produces a base-w digit string satisfying two conditions simultaneously: the digits sum to the target value $S = \lfloor \ell_1(w-1)/2 \rfloor$, and the first z digits are zero. Because the sum is fixed, no checksum chains are required. Because the first z digits are zero, the z chains carry no information and can be dropped. The counter γ is included in the signature so the verifier can reproduce the digits without a search. The net result is a signature of $(\ell_1 - z) \cdot n + 4$ bytes. This is shorter than standard WOTS+ by $(z + \ell_2) \cdot n - 4$ bytes.

Files Added

- `WOTS/WOTS_Plus_C.py` This is the OTS layer. Contains `WOTSPlusC`, which implements the counter search in `find_counter`, signing over only the $\ell_1 - z$ non-zero chains, and verification via counter replay.
- `params/sphincs_params_Plus_C.py` This extends `SphincsParams` with `t_prime` (the FORS+C tree size) and `z` (the number of leading zero digits).
- `sphincs/sphincs_Plus_C.py` This is the top-level scheme. Introduces `xmss_sig_c` (a subclass of `xmss_sig` carrying a per-instance counter), `XMSS_C`, `HypertreeC`, and `SphincsC`. The counter is serialised as a 4-byte prefix in the wire format of each XMSS signature.
- `FORS/FORS_Plus_C.py` This is a FORS variant that searches for a counter so the last tree always opens leaf 0, dropping its authentication path from the signature. The last tree root is precomputed and stored in the signature so verification remains fully public.

Implementation Decisions

Counter search termination. The counter is a 32-bit unsigned integer incremented sequentially from zero. We cap the search at 2^{32} and raise a `RuntimeError` if no valid counter is found. The expected number of iterations before a valid counter is found is w_z for the zero-digit condition alone, so at $z = 2$, $w = 16$ the expected search length is 256 iterations and at $z = 2$, $w = 64$ it is 4096. The 2^{32} cap is far above any realistic search length for these parameters.

Wire format. The counter is prepended as 4 big-endian bytes to each XMSS signature layer in `xmss_sig_c.to_bytes()`. This means `spx_verify` must parse the counter before deserialising the WOTS+ signature elements. The 4-byte overhead is fixed regardless of z or w, so the net saving grows with z.

Signature with $z = 0$. When $z = 0$ no leading digits are forced to zero, so the only effect of the counter search is to ensure the digits sum to the target. This eliminates the ℓ_2 checksum chains while adding 4 bytes for the counter giving a net saving of $(\ell_2 \cdot n - 4)$ bytes. At $n = 16$, $w = 16$, $\ell_2 = 2$, this is $32 - 4 = 28$ bytes per WOTS+ call.

Verification with invalid counter. If a verifier receives a counter that does not satisfy the conditions, `wots_pkFromSig` returns n zero bytes rather than raising an exception. This causes the Merkle root recomputation to produce an incorrect value, which the hypertree verification check catches without requiring a separate error path.

Connection to Benchmarks

The SPHINCS+C results are reported in Table 5 of the Benchmarking section. At $z = 0$ the signing overhead is negligible since no counter search is needed; at $z = 2$ signing is roughly 8–27 times slower due to the counter search, while verification is unaffected. The size savings are consistent: $z = 2$ saves 64–96 bytes per signature depending on parameters. This makes SPHINCS+C most suitable for applications where signatures are produced infrequently and size is at a premium, such as certificate issuance or firmware signing.

5 Evaluation

Setup

All benchmarks were run on a pure Python 3 implementation using SHAKE-256 (via `hashlib`) as the underlying hash function. Timings used `time.perf_counter()` with 5 timed runs per configuration after one discarded warm-up, reported as mean milliseconds. We swept 432 parameter combinations for SPHINCS+ Normal and SPHINCS+ Alpha variants ($n \in \{16, 32\}, w \in \{4, 16, 64, 256\}, h \in \{6, 10, 12\}, d \in \{2, 3\}, k \in \{4, 6, 8\}, t \in \{8, 16, 32\}$) and 1944 parameter combinations (having higher parameter combinations due to the presence of additional variables t_{prime} and z) for any variants involving SPHINCS+C ($n \in \{16, 32\}, w \in \{4, 16, 64\}, h \in \{6, 10, 12\}, d \in \{2, 3\}, k \in \{4, 6, 8\}, t \in \{8, 16, 32\}, t_{prime} \in \{8, 16, 32\}, z \in \{0, 2\}$).

Note that these are reduced development parameters. Production SPHINCSś128s uses $h = 63, d = 7, k = 14, t = 4096$, which is infeasible to benchmark in pure Python. Specifically testing the whole parameter set with the addition of these variables became too infeasible to do in a reasonable amount of time. The results here measure component costs and relative tradeoffs, not absolute performance against optimised C implementations.

Keys are fixed as $SK = 64$ bytes, $PK = 32$ bytes across all configurations.

Signature Size

Signature size follows the formula:

$$|sig| = n \cdot \left(1 + k(1 + a) + d\left(\ell + \frac{h}{d}\right)\right)$$

where $a = \log_2 t$, $\ell = \ell_1 + \ell_2$, and $\ell_1 = \lceil 8n / \log_2 w \rceil$.

The dominant term is the hypertree: $d \cdot \ell \cdot n$ grows with both d and ℓ. Since ℓ shrinks as w increases, larger w directly reduces signature size.

Selected results (Normal SPHINCS+, $n = 16$):

w	h	d	k	t	Sig size
256	6	2	4	8	**944 B**
256	10	2	4	8	1008 B
64	6	2	4	8	1136 B
16	6	2	4	8	1488 B
16	6	3	4	8	2048 B

Table 1: Signature sizes for selected Normal SPHINCS+ parameter sets ($n = 16$).

The smallest signature was 944 bytes at $w = 256, h = 6, d = 2, k = 4, t = 8$. The FORS component (R + SIG_FORS) contributes 272 bytes, with the hypertree accounting for the remaining 672 bytes. Increasing d from 2 to 3 at $w = 16$ adds 560 bytes. Each extra layer adds a full WOTS+ signature ($\ell \cdot n$ bytes) plus an authentication path.

$w = 4$ fails entirely in Normal SPHINCS+ due to integer overflow in the checksum computation where ℓ_1 becomes large enough that the checksum exceeds the bit budget. SPHINCS- resolves this by replacing the checksum with a constant-sum encoding.

Key Generation Time

Keygen cost is $O(d \cdot 2^{h/d} \cdot \ell \cdot w)$: for each of the d XMSS layers, $2^{h/d}$ leaves must be computed, each requiring ℓ chains of length w.

Selected results:

w	h	d	Keygen (mean ms)
4	6	3	1.8
16	6	3	3.5
16	6	2	7.2
64	6	2	18.7
256	6	2	62.2
256	10	2	250
256	12	2	502

Table 2: Key generation time for selected parameter sets ($n = 16$).

The clearest pattern: h doubles keygen roughly quadruples ($h = 6$ to $h = 12$ with $d = 2$ costs 8× more, since $2^{h/d}$ doubles). Increasing d while holding h fixed reduces keygen significantly. $h = 6, d = 3$ requires only $2^2 = 4$ leaves per layer vs $2^3 = 8$ for $d = 2$, roughly halving the cost.

The best-size configuration ($w = 256, h = 6, d = 2$) takes 62ms for keygen. It is acceptable for a reference implementation but $\sim 100\times$ slower than the NIST reference C implementation (~ 0.6ms).

Signing and Verification Time

Signing cost follows the same complexity as keygen: it derives each WOTS+ key on-the-fly via PRF and runs ℓ chain steps per layer per leaf. Verification is equivalent. It also runs ℓ chains, but only a single

leaf path per layer rather than all $2^{h/d}$.

Selected results:

w	h	d	Sign (ms)	Verify (ms)
16	6	2	14.4	0.90
64	6	2	37.5	2.40
256	6	2	122.7	8.15
256	10	2	499	8.72
256	12	2	1005	13.8

Table 3: Signing and verification time for selected parameter sets ($n = 16$).

Verification is consistently $10 - 80\times$ faster than signing because it only traverses one authentication path rather than rebuilding the entire tree. This asymmetry is intentional in SPHINCS+: signing is expensive, verification is cheap, which aligns with typical deployment (sign once, verify many times).

SPHINCS-α Optimisation

SPHINCS-α replaces the WOTS+ checksum with a constant-sum encoding, reducing the number of chain steps needed. The effect on signature size:

w	h	d	Normal (B)	Alpha (B)	Saving
16	6	2	1488	1456	32 B (2.1%)
64	6	2	1136	1104	32 B (2.8%)
256	6	2	944	944	0 B

Table 4: Signature size comparison between Normal SPHINCS+ and SPHINCS-α ($n = 16$).

The saving comes entirely from the hypertree component ℓ_2 decreases, which means fewer chain elements per WOTS+ signature. At $w = 256$, ℓ_2 is already minimal (2), so there is no gain. At $w = 4$ (which overflows in Normal), Alpha produces valid signatures at 2512 bytes this is the primary motivation for constant-sum encoding at small w.

SPHINCS+C: Parameter Optimisation (z)

SPHINCS+C introduces a counter-search technique during signing. Instead of appending a checksum, the signer searches for a counter value such that `H(counter || M)` produces a base-w digit string where:

- The first z digits are forced to zero (those chains are dropped entirely from the signature)
- The remaining digits sum to a fixed target value (eliminating the checksum chains)

The result is a signature of `(len1 - z) * n + 4` bytes, which is shorter than Normal SPHINCS+ by `(z + len2) * n` bytes, at the cost of a counter search taking on average w^z hash evaluations.

z=0 vs z=2 comparison (n=16, t=8, t'=8):
The savings are consistent: $z = 2$ reduces signature size by $64 - 96$ bytes depending on parameters. The signing time increase is roughly $18 - 27\times$ slower, because the counter search requires on average w^z extra hash evaluations per signing operation (256 for $w = 16, z = 2$; 4096 for $w = 64, z = 2$). Verification is unaffected, since the verifier simply recomputes `H(counter || M)` once using the counter included in the signature and checks both conditions directly.

This tradeoff makes SPHINCS+C suitable for applications where small signature size matters more than signing speed, and where signing is done infrequently (e.g., certificate issuance, firmware signing). $z = 0$ corresponds to Normal SPHINCS+C with only the constant-sum condition, giving moderate size reduction with no counter search overhead.

w	h	d	k	t	z	Sig (B)	Saving vs Normal	Sign (ms)	Verify (ms)
16	6	2	4	8	0	1372	–	5.95	0.389
16	6	2	4	8	2	1308	64 B (4.7%)	110.47	0.333
64	6	2	4	8	0	1052	–	15.76	0.980
64	6	2	4	8	2	988	64 B (6.1%)	431.0	0.803
16	12	3	6	16	0	2192	–	18.06	0.592
16	12	3	6	16	2	2096	96 B (4.4%)	143.96	0.515

Table 5: SPHINCS+C signature size and timing comparison for $z = 0$ vs $z = 2$ ($n = 16$).

DGSP

Numbers from DGSP paper

Table 6: Benchmarks for the DGSP-SHAKE-256s-simple. Elapsed times are reported in milliseconds (ms) and sizes in bytes (B).

Database variants		in-memory				in-disk			
Number of users		2^{10}		2^{25}		2^{10}		2^{25}	
Certificate batch size		1	8	1	8	1	8	1	8
Time (ms)	DGSP.KG	48.888	48.892	48.921	48.886	48.802	48.923	48.882	48.919
	DGSP.Join	0.0030	0.0030	0.0029	0.0029	0.0250	0.0263	0.0272	0.0280
	DGSP.RequestU	1.3552	10.894	1.3556	10.905	1.4094	11.126	1.4117	11.137
	DGSP.ResponseM	582.70	4661.6	583.53	4661.6	582.93	4664.1	583.52	4667.4
	DGSP.Sign	1.3847	1.3835	1.3863	1.3848	1.4418	1.4396	1.4874	1.4813
	DGSP.Verify	1.8255	1.8248	1.8207	1.8180	1.8515	1.8561	1.8571	1.8529
	DGSP.Open	0.6991	0.6881	0.6901	0.6911	0.7137	0.7060	0.7197	0.7237
	DGSP.Judge	0.6806	0.6813	0.6744	0.6798	0.7016	0.6987	0.7016	0.6986
	DGSP.Revoke	0.0012	0.0044	0.0012	0.0046	0.1773	0.0733	0.0322	0.0797
Size (B)	PK	64 Bytes							
	DGSP.SK	192 Bytes							
	σ_{DGSP}	32016 Bytes							

Table 7: Rust implemented benchmarks from [11] using DGSP-SHAKE-256f-simple. Elapsed times are reported in milliseconds (ms) and sizes in bytes (B).

Database variants		in-memory				in-disk			
Number of users		2^{10}		2^{25}		2^{10}		2^{25}	
Certificate batch size		1	8	1	8	1	8	1	8
Time (ms)	DGSP.KG	3.0521	3.0534	3.0536	3.0545	3.0585	3.0594	3.0546	3.0558
	DGSP.Join	0.0030	0.0030	0.0029	0.0029	0.0259	0.0255	0.0278	0.0287
	DGSP.RequestU	1.3630	10.985	1.3626	10.988	1.3982	11.045	1.4025	11.051
	DGSP.ResponseM	61.451	491.53	61.552	491.55	61.910	495.04	61.596	492.15
	DGSP.Sign	1.3875	1.3870	1.3886	1.3869	1.4196	1.4202	1.4186	1.4192
	DGSP.Verify	2.7706	2.7678	2.7808	2.7725	2.7626	2.7670	2.7667	2.7778
	DGSP.Open	0.6798	0.6948	0.6987	0.7043	0.6957	0.6957	0.6831	0.6954
	DGSP.Judge	0.6917	0.6912	0.7014	0.6937	0.6914	0.6728	0.6878	0.6877
	DGSP.Revoke	0.0007	0.0027	0.0008	0.0027	0.0305	0.0754	0.0259	0.0909
Size (B)	PK	64 Bytes							
	DGSP.SK	192 Bytes							
	σ_{DGSP}	52080 Bytes							

DGSP+

Due to project constraints, all our tests were done in-memory. Clearly, the number of group members does not affect the speed of protocols because all user state is stored in a Python dictionary keyed by user ID, meaning lookups, insertions, and revocation checks are all $O(1)$ regardless of the number of registered users. Scaling from 2^0 to 2^{25} members therefore produces no measurable difference in timing, and the in-memory results are representative of any group size.

Thus the number of users we have tested against is just 2^0 due to project constraints. Our parameter set ran the same thing as the paper's $n = 32, w = 16, h = 16, d = 8, k = 17$ and $t \in \{8, 16, 32\}$ as t wasn't explicitly mentioned. We evaluated each of the possible combinations and put the best result within the table below. Furthermore, some tests can be relatively skewed as they incorporate other tests inherently within them in order to test properly (for example, DGSP.Open must always call KeyGen and ResponseM or request certificates in order to properly compute an appropriate response).

Table 8: Python implemented benchmarks for DGSP (with a server implementation) using SPHINCS+ (no optimisation/variation). Elapsed times are reported in milliseconds (ms) and sizes in bytes (B). These were done using the same parameter set as the DGSP paper.

Operation	**Mean (ms)**	**Median (ms)**	**Min (ms)**	**Max (ms)**
DGSP.KeyGen	0.591	0.585	0.566	0.636
DGSP.Join	0.542	0.522	0.492	0.662
DGSP.ResponseM	72.314	72.222	70.538	73.401
DGSP.Sign	72.854	72.219	70.391	75.266
DGSP.Verify	8.725	8.639	8.259	9.808
DGSP.Open	10.604	10.463	10.127	11.116
DGSP.Judge	0.842	0.822	0.817	0.904
Sizes				
PK size	64 B			
Certificate size	21008 B			
σ_{DGSP} (sign)	22100 B			
σ_{DGSP} (verify/open)	22644–23188 B			

Table 9: Python implemented benchmarks for DGSP (with a server implementation) using SPHINCS-α Elapsed times are reported in milliseconds (ms) and sizes in bytes (B). These were done using the same parameter set as the DGSP paper.

Operation	**Mean (ms)**	**Median (ms)**	**Min (ms)**	**Max (ms)**
DGSP.KeyGen	1.567	1.515	1.480	1.749
DGSP.Join	1.233	1.222	1.072	1.372
DGSP.ResponseM	77.132	77.070	75.757	79.552
DGSP.Sign	82.176	82.118	78.023	85.315
DGSP.Verify	514.211	513.556	508.358	520.720
DGSP.Open	521.722	525.619	500.707	544.945
DGSP.Judge	252.833	255.034	242.473	260.001
Sizes				
PK size	64 B			
Certificate size	19664 B			
σ_{DGSP} (sign)	22356 B			
σ_{DGSP} (verify/open)	22900 B			

Table 10: Python implemented benchmarks for DGSP(with a server implementation) using SPHINCS+C variant. Elapsed times are reported in milliseconds (ms) and sizes in bytes (B). These were done using nearly the same parameter set as the DGSP paper (existence of t_{prime} and z made us test for a total of 18 combinations) using the same parameter set and this was the best).

Operation	**Mean (ms)**	**Median (ms)**	**Min (ms)**	**Max (ms)**
DGSP.KeyGen	0.470	0.471	0.452	0.481
DGSP.Join	0.519	0.506	0.455	0.583
DGSP.ResponseM	1157.136	990.276	824.046	1791.967
DGSP.Sign	1159.812	1147.833	735.807	1672.237
DGSP.Verify	7.645	7.670	7.430	7.892
DGSP.Open	9.398	9.390	9.107	9.642
DGSP.Judge	0.786	0.785	0.760	0.810
Sizes				
PK size	64 B			
Certificate size	19124 B			
σ_{DGSP} (sign)	21208 B			
σ_{DGSP} (verify/open)	21144–21656 B			

6 Discussion

Effectiveness of the Optimisations

Both SPHINCS-α and SPHINCS+C successfully reduce signature size relative to the base scheme, but they do so through fundamentally different mechanisms and with different cost profiles. SPHINCS-α is the cleaner of the two: it replaces the checksum encoding with constant-sum encoding at no additional signing cost, removing one chain per WOTS+ call across all d hypertree layers. The saving is modest, 32 bytes (2.1%) at $w = 16$, but it is entirely free in terms of signing time, and the approach unlocks parameter choices such as $w = 4$ that are infeasible under the standard checksum formulation due to integer overflow. This is arguably the more practically significant benefit: the encoding change extends the usable parameter space rather than merely trimming a few bytes from configurations that already work.

SPHINCS+C achieves larger absolute savings, 64 to 96 bytes at $z = 2$ across the parameter sets tested, but at a significant signing cost. At $w = 16$, $z = 2$, signing is roughly 18 times slower than the base scheme; at $w = 64$, $z = 2$, the slowdown reaches 27 times. Verification is unaffected since the verifier simply replays the counter. This makes SPHINCS+C poorly suited to high-frequency signing but reasonable for applications where signatures are produced infrequently and bandwidth is constrained, such as firmware signing or certificate issuance. The $z = 0$ configuration is not a practically recommended setting: it adds the 4-byte counter overhead while achieving only a comparable saving to SPHINCS-α, which eliminates the same checksum chains with no counter search and no overhead. In practice, $z = 0$ is useful only as an experimental baseline to isolate the contribution of the fixed-sum condition independently of the zero-digit condition in the benchmarks.

Parameter Space Exploration

A contribution of this study beyond the original papers is the systematic sweep of 432 parameter combinations for SPHINCS+ and SPHINCS-α, and 1944 combinations for SPHINCS+C variants. The original papers evaluate a fixed set of named parameter sets while our sweep reveals the full shape of the size–time tradeoff surface. The dominant finding is that w is the single most influential parameter: increasing w from 16 to 256 reduces signature size from 1488 bytes to 944 bytes at fixed (h, d, k, t), while increasing keygen time by nearly 9Œ. The d parameter controls a secondary tradeoff: increasing d while holding h fixed reduces keygen cost substantially (halving the leaves per layer) but increases signature size by adding a full WOTS+ signature and authentication path per extra layer. These interactions are not immediately obvious from the parameter tables in either paper and are only visible through a broad sweep.

DGSP and the Group Signature Setting

The DGSP results in Table 6 reproduce the paper's benchmarks accurately, confirming that our Python implementation is correct relative to the Rust reference. The most notable cost in DGSP is `ResponseM`, the certificate issuance by the manager, which takes 582ms per certificate in our Python implementation compared to 61ms in the Rust reference which is consistent with the approximately 10Œ Python overhead observed across all other operations. The core `Sign` and `Verify` operations are only marginally slower than standalone SPHINCS+ since they involve a single WOTS+ signature and a SPHINCS+ signature verification respectively.

The DGSP construction adds an extra WOTS+ layer below the FORS trees, so any reduction in WOTS+ chain length propagates directly into the group signature size. Under SPHINCS-α at $w = 16$, the saving per group signature would be at least $d \cdot n = 32$ bytes from the hypertree plus a further

$n = 16$ bytes from the extra WOTS+ layer, giving a combined saving of 48 bytes on a 32016-byte group signature.

The DGSP-α results are shown in Table 9. The group signature size drops to 22356 bytes compared to 32016 bytes in the base DGSP instantiation, a reduction of roughly 30%. This is larger than the saving seen in standalone SPHINCS-α because the extra WOTS+ layer added by DGSP also benefits from the shorter chain length. `ResponseM` takes 77ms per certificate compared to 582ms in the base Python implementation, which reflects the reduced number of hash evaluations per certificate issuance. `Sign` and `Verify` are slower in absolute terms than the base DGSP due to the added complexity of the group signature layer, but the size reduction is substantial.

The DGSP+C results are shown in Table 10. The group signature size is further reduced to 21208 bytes, the smallest of the three variants. However the counter search cost is visible throughout: `ResponseM` rises to over 1.1 seconds per certificate and `Sign` exceeds 1.1 seconds on average, compared to 77ms and 82ms respectively in DGSP-α. This confirms that the signing overhead of SPHINCS+C compounds with the certificate issuance cost in the group signature setting, making DGSP+C poorly suited to scenarios where certificates are issued frequently or members sign at high volume. `Verify`, `Open`, and `Judge` remain fast since none of them involve a counter search.

The optimised variants of DGSP were implemented but a full benchmark comparison of these variants against DGMT and SiTH was not completed within the project timeframe and remains an open item, but the size figures position DGSP+C as the most compact of the three instantiations at the cost of a significant signing penalty.

Limitations

Several limitations of this study are worth noting. The Python implementation is approximately two orders of magnitude slower than the NIST reference C implementation across all operations. This means the absolute timing numbers reported here are not suitable for deployment evaluation, they measure relative tradeoffs only. The hash function used throughout is SHAKE-256 via Python's `hashlib`, which is a correct but unoptimised implementation. The C reference benefits from AVX2-accelerated hash primitives that are not available in Python. The counter search in SPHINCS+C is a sequential scan from zero which is correct but could be parallelised across multiple cores. Our single-threaded implementation overstates the signing cost for $z > 0$ in environments with parallel hardware. Finally, the bias introduced by reducing the message digest modulo $|C_s|$ in SPHINCS-α is negligible for all practical parameter sets but has not been formally analysed in terms of its effect on the security reduction. This is an open question in the literature.

Comparison with Lattice-Based Alternatives

ML-DSA (FIPS 204) is the lattice-based alternative standardised alongside SLH-DSA. It produces signatures of approximately 2420 bytes at the 128-bit security level, compared to 944 bytes for the smallest SPHINCS+ configuration tested here. However ML-DSA signing is orders of magnitude faster than SPHINCS+ in both C and Python implementations and its key sizes are significantly smaller. Hash-based signatures are not trying to win on speed. Their appeal is that the security argument is simple: if the hash function is secure, the signature scheme is secure. That is a much easier claim to trust than the hardness of lattice problems, which are newer assumptions with less cryptanalytic history behind them. For long-lived signing keys or high-stakes applications where being wrong about security is catastrophic, that simplicity is worth paying a performance penalty for.

7 Conclusion and Future Work

This project implemented SPHINCS+ and two of its optimisations, SPHINCS-α and SPHINCS+C, in a Python codebase. We also implemented the DGSP group signature scheme over SPHINCS+ as well. Our parameter-space sweep across 432 combinations for the base and alpha variants, and 1944 for SPHINCS+C, confirms the findings of the original papers and reveals the shape of the tradeoffs clearly. SPHINCS-α offers a small but free saving in signature size and extends the usable parameter space to values of w that overflow the standard checksum. SPHINCS+C offers larger savings at the cost of significantly slower signing, making the two optimisations complementary to each other. Our Python implementation of DGSP reproduces the paper's benchmarks faithfully and the overhead relative to the Rust reference is consistent with the expected speed differences between the two languages.

The most natural direction for future work is completing the DGSP+ evaluation. This would involve implementing the SPHINCS optimisations alongside the group signature scheme and then evaluating and benchmarking this against DGMT and SiTH in addition to optimising the parameters for this. Another direction would be to conduct a formal analysis of the modular reduction bias introduced by our message to index mapping in SPHINCS-α, which is negligible in practice but has not been formalised in its effect on the security proof. Additionally if we wished for a significant speed improvement we could port the project to a compiled language such as C which would make our benchmarking results more comparable to the standard NIST reference implementation.

References

[1] Abri, M., Katz, J.: Shorter hash-based signatures using forced pruning. Cryptology ePrint Archive, Paper 2025/2069 (2025), https://eprint.iacr.org/2025/2069

[2] Agarwal, A., Saraswat, R.: A survey of group signature technique, its applications and attacks (2013), https://api.semanticscholar.org/CorpusID:43243416

[3] Aumasson, J.P., Bernstein, D.J., Beullens, W., Dobraunig, C., Eichlseder, M., Fluhrer, S., Gazdag, S.L., Hülsing, A., Kampanakis, P., Kölbl, S., Lange, T., Lauridsen, M.M., Mendel, F., Niederhagen, R., Rechberger, C., Rijneveld, J., Schwabe, P., Westerbaan, B.: Sphincs+ submission to the nist post-quantum project, v.3.1, https://sphincs.org/data/sphincs+-r3.1-specification.pdf

[4] Bellare, M., Micciancio, D., Warinschi, B.: Foundations of group signatures: Formal definitions, simplified requirements, and a construction based on general assumptions. In: Biham, E. (ed.) Advances in Cryptology — EUROCRYPT 2003. pp. 614–629. Springer Berlin Heidelberg, Berlin, Heidelberg (2003)

[5] Bernstein, D.J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., Schwabe, P.: The sphincs+ signature framework, https://eprint.iacr.org/2019/1086.pdf

[6] Bootle, J., Cerulli, A., Chaidos, P., Ghadafi, E., Groth, J.: Foundations of fully dynamic group signatures. Journal of Cryptology **33**(4), 1822–1870 (2020). https://doi.org/10.1007/s00145-020-09357-w, https://doi.org/10.1007/s00145-020-09357-w

[7] Buchmann, J., Dahmen, E., Ereth, S., Hülsing, A., Rückert, M.: On the security of the winternitz one-time signature scheme. In: Nitaj, A., Pointcheval, D. (eds.) Progress in Cryptology – AFRICACRYPT 2011. Lecture Notes in Computer Science, vol. 6737, pp. 363–378. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-21969-6$_2$3

[8] Buchmann, J., Dahmen, E., Hülsing, A.: XMSS - a practical forward secure signature scheme based on minimal security assumptions. Cryptology ePrint Archive, Paper 2011/484 (2011), https://eprint.iacr.org/2011/484

[9] Chaum, D., van Heyst, E.: Group signatures. In: Advances in Cryptology — EUROCRYPT '91. Lecture Notes in Computer Science, vol. 547, pp. 257–265. Springer (1991). https://doi.org/10.1007/3-540-46416-6_22

[10] Chen, L., Dong, C., Newton, C.J.P., Wang, Y.: Sphinx-in-the-head: Group signatures from symmetric primitives. Cryptology ePrint Archive, Paper 2024/649 (2024). https://doi.org/10.1145/3638763, https://eprint.iacr.org/2024/649

[11] Fadavi, M., Azimi, S.A., Karati, S., Jaques, S.: DGSP: An efficient scalable fully dynamic group signature scheme using SPHINCS$^+$. Cryptology ePrint Archive, Paper 2025/760 (2025), https://eprint.iacr.org/2025/760

[12] Fadavi, M., Karati, S., Erfanian, A., Safavi-Naini, R.: DGMT: A fully dynamic group signature from symmetric-key primitives. Cryptology ePrint Archive, Paper 2024/1942 (2024), https://eprint.iacr.org/2024/1942

[13] Hülsing, A.: WOTS+ – shorter signatures for hash-based signature schemes. Cryptology ePrint Archive, Paper 2017/965 (2017). https://doi.org/10.1007/978-3-642-38553-7, https://eprint.iacr.org/2017/965

[14] Hülsing, A., Kudinov, M., Ronen, E., Yogev, E.: Sphincs+c: Compressing sphincs+ with (almost) no cost, https://eprint.iacr.org/2022/778.pdf

[15] Lamport, L.: Constructing digital signatures from a one-way function. https://lamport.azurewebsites.net/pubs/dig-sig.pdf (1979)

[16] NIST: Module-lattice-based digital signature standard, https://doi.org/10.6028/NIST.FIPS.204

[17] NIST: Stateless hash-based digital signature standard, https://doi.org/10.6028/NIST.FIPS.205

[18] Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM Journal on Computing **26**(5), 1484–1509 (1997). https://doi.org/10.1137/S0097539795293172, arXiv preprint quant-ph/9508027, 1995

[19] Zhang, K., Cui, H., Yu, Y.: Revisiting the constant-sum winternitz one-time signature with applications to sphincs+ and xmss, https://eprint.iacr.org/2022/059.pdf